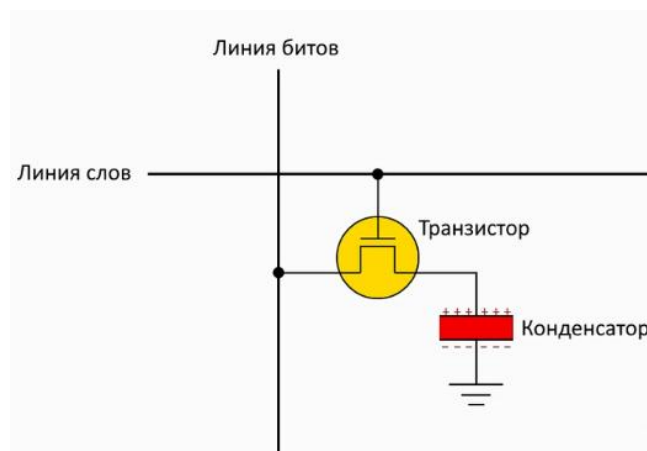
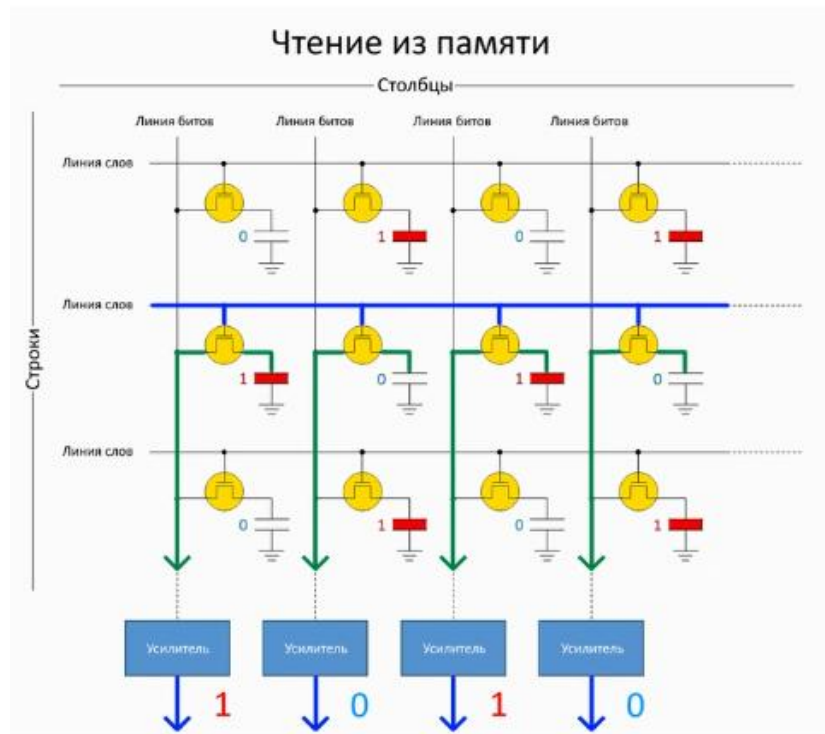
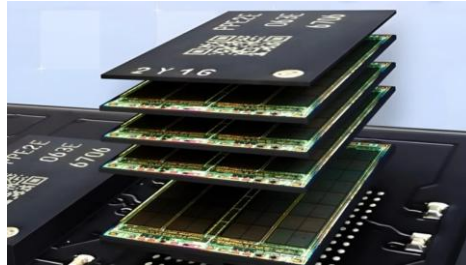
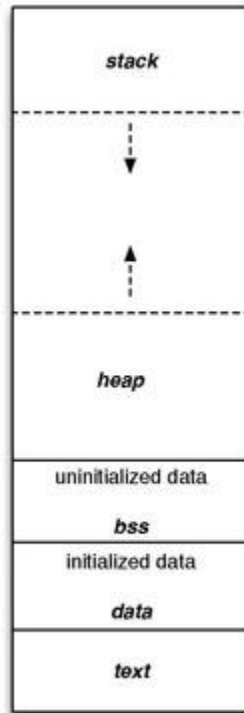


Data structures

What is RAM and how it works?





(https://en.wikipedia.org/wiki/Data_segment#:~:text=modern%20programming%20languages.-,Program%20memory,-%5Bedit%5D)

ARRAY

There are arrays declared statically and arrays declared dynamically. All arrays are references. The value of an array is its address. Arrays subscripts start at zero. In C++, arrays do not know how many elements they have.

Statically declared arrays

Statically declared arrays are allocated memory at compile time and their size is fixed, i.e., cannot be changed later.

For example two int arrays are declared, one initialized, one not.

```
int a[10];  
int b[5] {8, 20, 25, 9, 14};
```

The `a` array has 10 elements with subscripts numbered from 0 to 9, filled with garbage. The `b` array has 5 elements with subscripts numbered from 0 to 4, filled with the five given values. They accessed in the usual way, e.g., `a[5]` or `b[2]`. The memory is allocated at compile time. A memory picture of these arrays show 10 int garbage values and 5 valid int values:

```
//
//  a| -|-->|...|...|...|...|...|...|...|...|...|
//
//      0  1  2  3  4  5  6  7  8  9
//
//  b| -|-->| 8 | 20| 25| 9 | 14|
//
//      --- --- --- --- ---
```

Static multi-dimensional arrays are declared with multiple dimensions. For example, a 2-dimensional array, `c`, has 3 rows and 4 columns:

```
// 2 dimensional array 3x4
int c[3][4];

//
//  c| -|--> 0|...|...|...|
//
//      --- --- --- ---
//
//      1|...|...|...|
//
//      --- --- --- ---
//
//      2|...|...|...|
//
//      --- --- --- ---
//
//      0  1  2  3
```

Dynamically declared arrays

If you want to be able to alter the size of your array at run time, then declare dynamic arrays. These are done with pointers and the `new` operator. (<https://courses.washington.edu/css342/zander/css332/pointers.html>)

Allocate memory using `new`, and then you access the array in the same way you would a static array. For example,

```
// Dynamic array allocation and access
int* arrayPtr = new int[10];
for (int i = 0; i < 10; i++) {
    arrayPtr[i] = i;
}
```

The memory picture is identical to the static array, but you can change the size if you need to. Don't forget you must deallocate the memory before allocating new memory (or you will have a memory leak).

```
// Don't forget you must deallocate the memory
delete [] arrayPtr; // the [] is needed when deleting array pointers
arrayPtr = new int[50];
```

A **static array**, depending on how and where it is declared, may reside either on the **stack** or in the **data segment**.

```
void foo() {
    // In this case, arr is allocated on the stack
    int arr[100];
}
```

Allocation here is lightning-fast and automatically cleaned up, but — and here lies the rub — stack memory is quite limited. Typically, one ought not to place large arrays here, lest one risk a most undignified **stack overflow**.

```
// These are placed in the data segment (.data for initialized or .bss for uninitialized)
void foo2() {
    static int arr1[100]; // inside a function
}
int arr2[100];           // outside any function
```

These are placed in the data segment — more precisely, either in the .data (for initialized) or .bss (for uninitialized) sections of memory. Their lifetime spans the entire duration of the program, and they are initialized either at compile time or zero-initialized by the runtime.

A dynamic array, declared via new, is allocated on the heap — a large region of memory reserved for objects with dynamic lifetime:

```
int* arr = new int[100]; // allocated on the heap
```

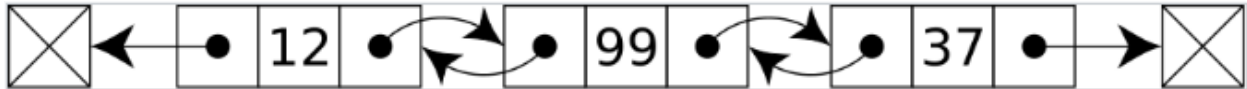
Declaration Type	Memory Location	Lifetime
int arr[10]; (local)	Stack	Until end of scope
static int arr[10];	Data segment	Entire program duration
int arr[10]; (global)	Data segment	Entire program duration
new int[10];	Heap	Until delete[] called

Linked List

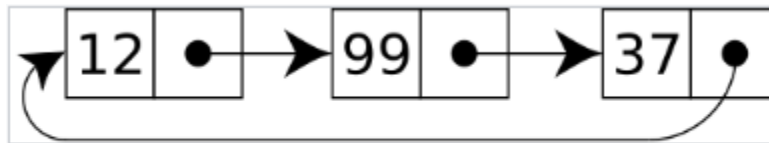
In computer science, a linked list is a linear collection of data elements whose order is not given by their physical placement in memory. Instead, each element points to the next. It is a data structure consisting of a collection of nodes which together represent a sequence.



A linked list is a sequence of nodes that contain two fields: data (an integer value here as an example) and a link to the next node. The last node is linked to a terminator used to signify the end of the list.



A doubly linked list whose nodes contain three fields: an integer value, the link forward to the next node, and the link backward to the previous node



A circular linked list

	Peek (index)	Mutate (insert or delete) at ...			Excess space, average
		Beginning	End	Middle	
Linked list	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$, known end element; $\Theta(n)$, unknown end element	$\Theta(n)$	$\Theta(n)$

VECTOR